SPEAKER

Pratik Pathak

Microsoft Certified Trainer

in/pathakpratik/ pratikpathak.exe zpratikpathak pratikpathak.com

HI 

Question?

Have you ever used Agents?

Why??

THE REALITY OF CONVERSATIONAL AI

The Anti-Pattern: Developers as 'Human Routers'

Conversational UI in an IDE creates friction, context drift, and prompt fatigue.

❌ THE CONVERSATIONAL TRAP

- Developer writes ambiguous prose prompt in IDE chat panel.
- Copilot generates plausible code with subtle edge-case bugs.
- Developer runs test suite manually, sees 40-line stack trace.
- Developer copies error into chat: 'Please fix line 42 null error.'
- Context window pollutes; circular debugging burns hours.
- Outcome: High cognitive fatigue & non-deterministic code.

✓ THE AUTONOMOUS FEEDBACK HARNESS

- Developer defines strict invariant tests and type contracts.
- Harness triggers headless Copilot Agent via CLI / Agent API.
- Compiler & test runner execute inside sandboxed evaluation loop.
- Harness extracts structured AST diagnostics from test failures.
- Automated reprompt payload feeds directly back into the agent.
- Outcome: 100% green build verified before human reviews diff.

EMPIRICAL BOTTLENECKS

The True Cost of Manual Prompting

Why treating an autonomous AI coding assistant like a chatbot fails at scale.

40%

DEVELOPER TIME WASTED

Spent acting as 'glue': copying traces, formatting chats, and micromanaging suggestions.

68%

TOKEN CONSUMPTION SAVINGS

Achieved by replacing massive conversational histories with isolated AST failure vectors.

< 1.2%

POST-COMMIT BUG RATE

Down from 31% by using deterministic test gates before human code review.

in/pathakpratik/

zpratikpathak

FOUNDATION LAYER 1

Context Engineering: Immutable Repository Rules

You cannot automate what you cannot constrain. Set boundaries in `.github/copilot-instructions.md`.

THE INVISIBLE FENCE

- Context Engineering is passive context injection.
- Instead of explaining conventions in every chat, write machine-readable rules directly into the repo root.
- GitHub Copilot Agent inherits these instructions on every single invocation without developer effort.
- Guarantees the agent stays inside domain boundaries and never mutates protected contracts.

 # .github/copilot-instructions.md (Actual Demo Guardrails)

```
# 1. Code Quality & Typing
- Always write strict, idiomatic Python with typing hints.
- Never alter public method signatures or return types in src/.

# 2. Boundary Constraints
- Never modify files inside tests/ directory.
- Keep function cyclomatic complexity <= 8.

# 3. Concurrency Safety
- Ensure all asynchronous shared-state operations use asyncio.Lock.
- Never release locks before background I/O operations resolve.
```

FOUNDATION LAYER 2

The Compiler & Test Suite as the Ultimate Prompt

Natural language is inherently ambiguous; strong type systems and invariant tests are mathematically exact.

PROSE PROMPT (FUZZY & SLOW)

```
Prompt: "Hey Copilot, please make sure the distributed lock class
is thread-safe and does not have race conditions when tasks arrive
quickly."
```

⚠️ What does 'thread-safe' mean to the LLM? Threading? Asyncio? Multiprocessing?

⚠️ How does the AI know it succeeded? It does not.

⚠️ Result: Plausible-looking code that still races under load.

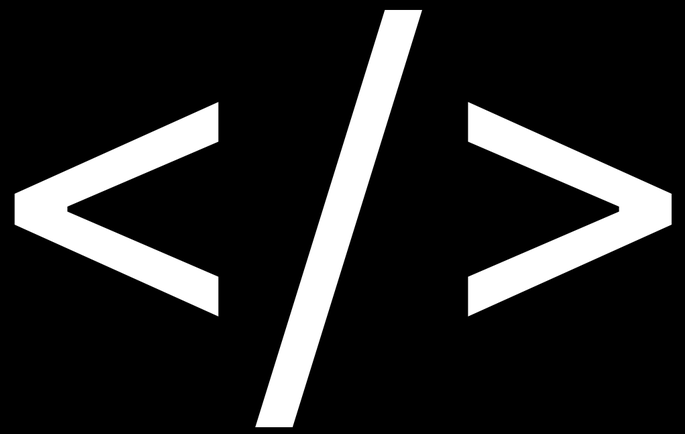
INVARIANT TEST CONTRACT (EXACT & BINARY)

```
Test Contract:
await asyncio.gather(
    lock.mutate("A", delay=0.06),
    lock.mutate("B", delay=0.02),
    lock.mutate("C", delay=0.04)
)
assert lock.get_state() == "update-C"
```

✓ Binary Truth: Status is strictly PASS or FAIL.

✓ Generates exact AST diagnostics when violated.

✓ Provides an irrefutable finish line for autonomous agent iteration.



DEEP DIVE

Inside the Harness: The Failure Vector Synthesizer

Turning 80 lines of terminal error noise into a high-density, low-token AI prompt payload.

RAW TERMINAL NOISE (~1,200 TOKENS)

```
================ FAILURES ================
__ test_concurrent_mutations_preserve_order __
Traceback (most recent call last):
  File "site-packages/_pytest/runner.py", line 341
  File "site-packages/pluggy/_hooks.py", line 513
  File "site-packages/pytest_asyncio.py", line 124
  [... 75 lines of internal library calls ...]
E   AssertionError: assert 'update-A' == 'update-C'
E     - update-C
E     + update-A
tests/test_lock.py:18: AssertionError
```

SYNTHESIZED HARNESS PAYLOAD (85 TOKENS)

```
Task: Fix concurrency race condition in src/distributed_lock.py

Failure Vector:
AssertionError: assert 'update-A' == 'update-C'
Location: tests/test_lock.py:18
Diagnostic: Asyncio coroutine executed without sequential lock
barrier.

Constraint:
Return ONLY updated Python class using asyncio.Lock.
Do not alter public method signatures.
```

Intelligence alone doesn't compound

01

01

Cold start

Every task begins from zero; last run's hard-won context is gone.

02

Domain-specific

What makes an agent brilliant at your work won't just emerge from a bigger model.

03

No compounding

Without memory, task 50 is no better than task 1.

How agent memory evolved

The lessons so far

Three throughlines across every memory approach that stuck.

Format

Markdown works — everything that stuck relies on human-readable context.

```
.github/copilot-instructions.md · skills · memory/*.md
```

Reading

Load progressively — frontmatter and search pull in only what's relevant, never everything at once.

```
frontmatter + grep, not the whole store
```

Writing

Autonomy wins — agents do best when free to record what they deem relevant, how they see fit.

```
the agent decides what's worth keeping
```

BOTTLENECK 02 · PRODUCTIONISING MEMORY

Scaling memory in production

02

01

Concurrent writes

Many agents edit the same memory files at the same time.

02

Lost attribution

Who wrote which memory — you or the agent — and when?

03

Stale & mixed scopes

Org knowledge and working notes tangle; old context pollutes runs.

in/pathakpratik/

zpratikpathak

SCALING TO PRODUCTION

Versioning & concurrency

01

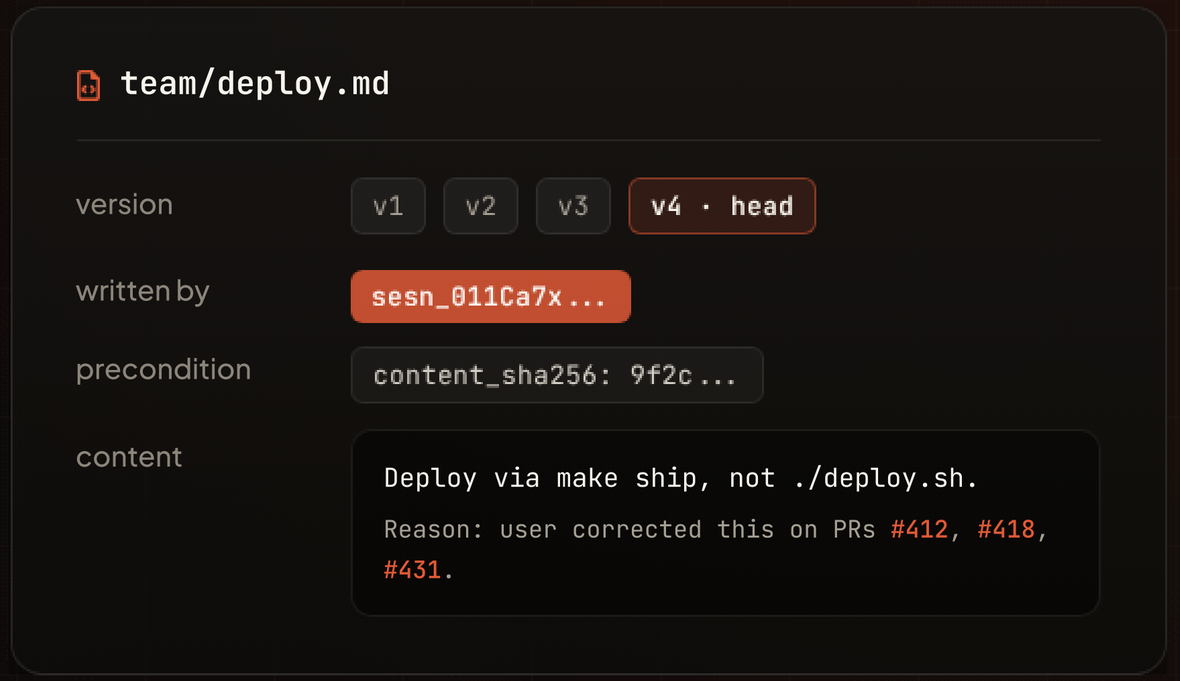
Versioning

Every write is attributed to an author, a session, a time — full history to inspect and roll back.

02

Concurrency

A content-hash precondition before each write — living in the harness, not the model.



SCALING TO PRODUCTION

Built for multi-agent systems

03

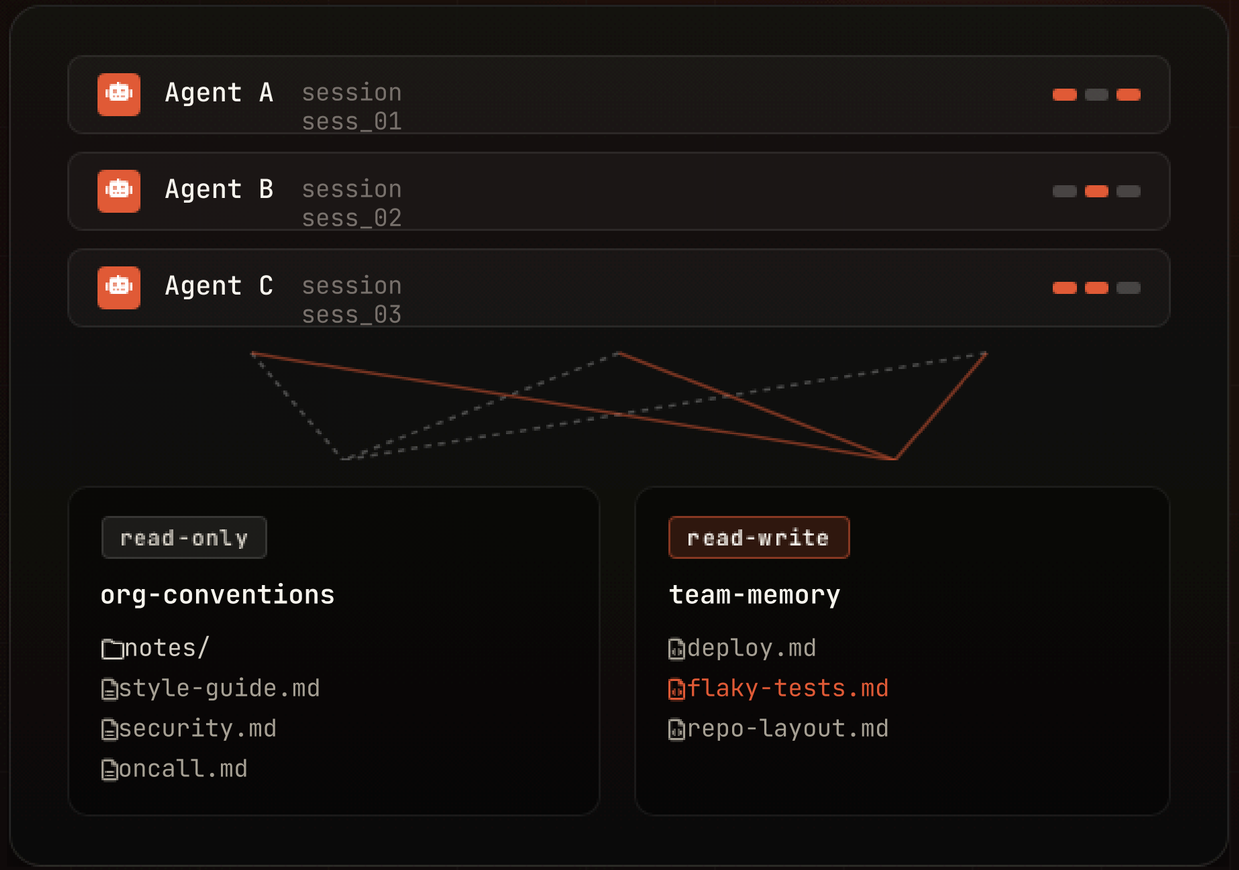
Permissioning

Read-only access to shared org knowledge; read-write to an agent's own working memory.

04

Portability

It's just files, with a standalone API on top — memory goes where you go.



IN PRODUCTION

What teams see with memory

97% fewer first-pass errors

"Memory lets us put continuous learning into production at scale — at 27% lower cost and 34% lower latency, and learning stays under our control."

GM, AI for Business
Global commerce platform

30% faster verification

"In our document-verification pipeline, cross-session memory lets agents remember common issues — including ones we didn't think about."

Head of Machine Learning
Document-AI platform

Focus on the product

"A lot of our work is making sense of fast-moving, messy conversations between teams and their agents. Memory lets us stop building memory infrastructure and focus on the product itself."

Founder
Early-stage AI startup

BOTTLENECK 03 · MEMORY OVER TIME

In-band memory reaches limits

03

01

Split focus

Every session the agent divides attention between finishing the task and maintaining memory — a difficult optimisation.

02

Patterns obfuscated

Agents writing to memory in-band miss patterns from across sessions and across different agents.

03

Memories go stale

Duplicates that disagree, notes gone stale — confidently wrong.

in/pathakpratik/

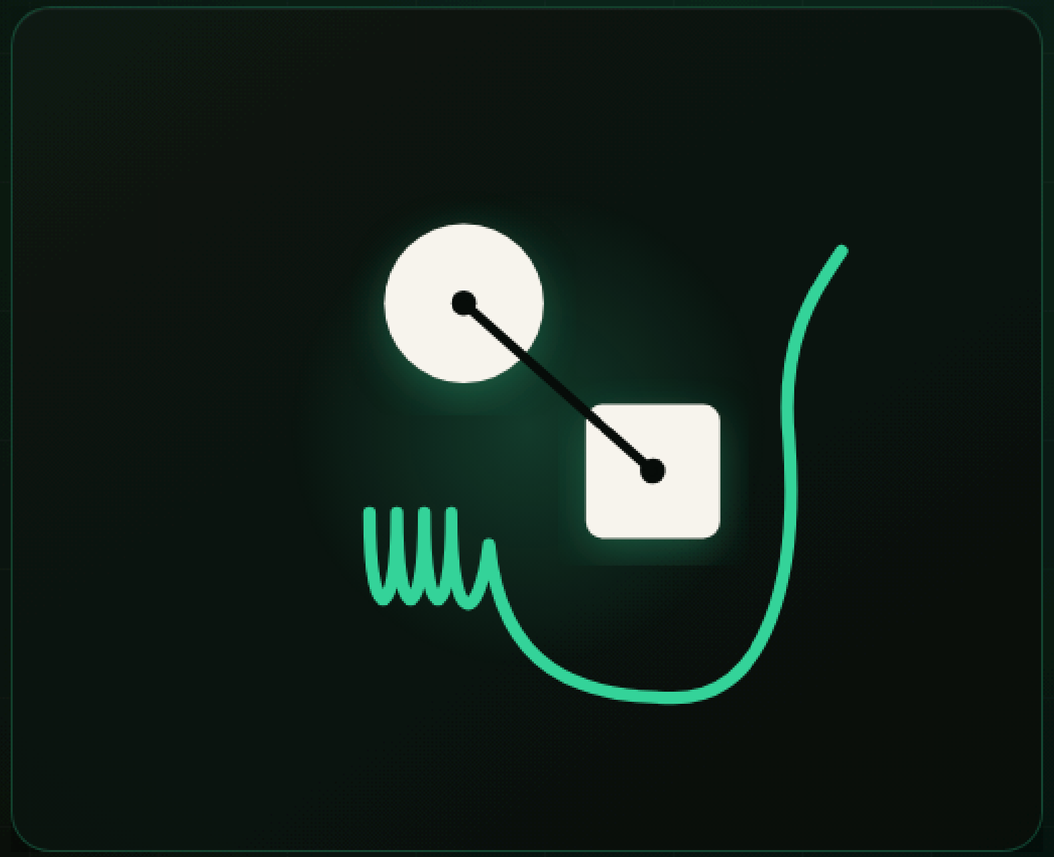
zpratikpathak

THE RESOLUTION · A SECOND-ORDER PROCESS

Introducing Dreaming

A batch process that runs out of band — with a single objective of curating memory.

`Out-of-band memory maintenance`

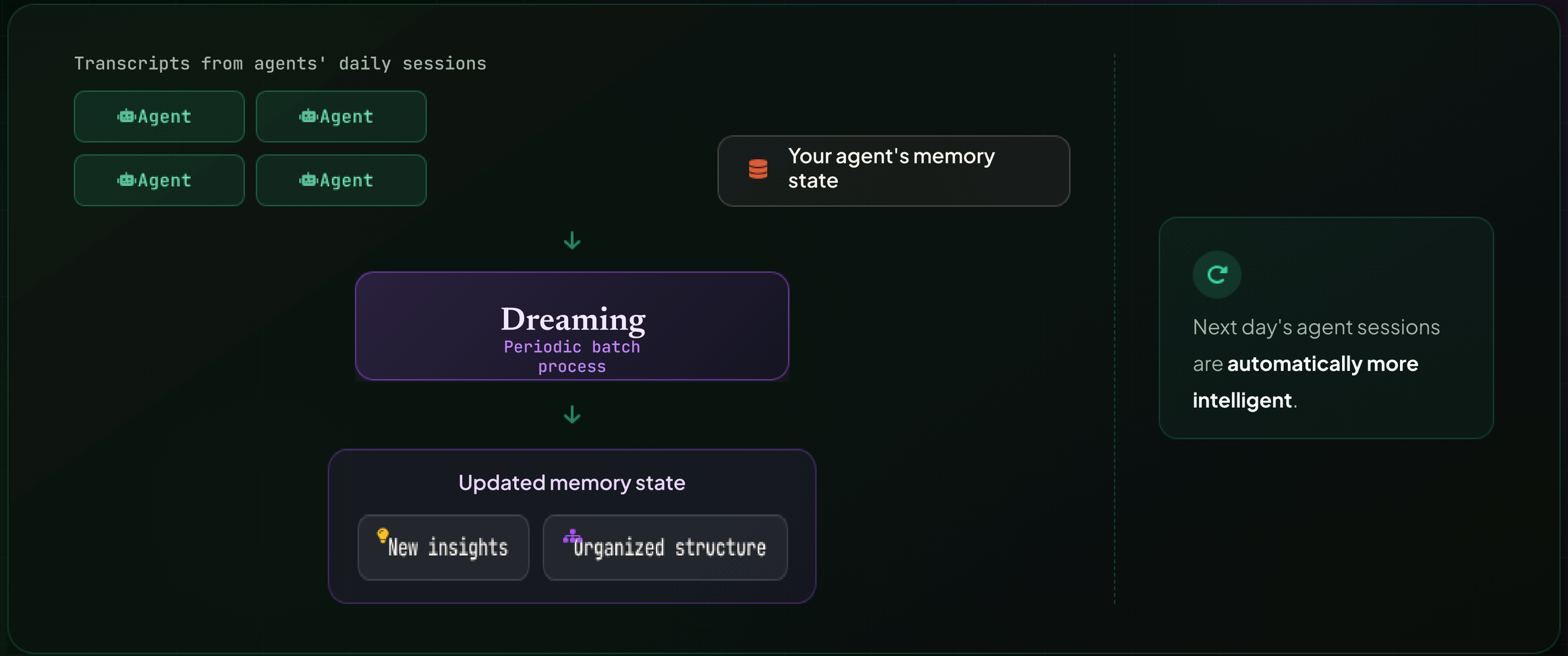


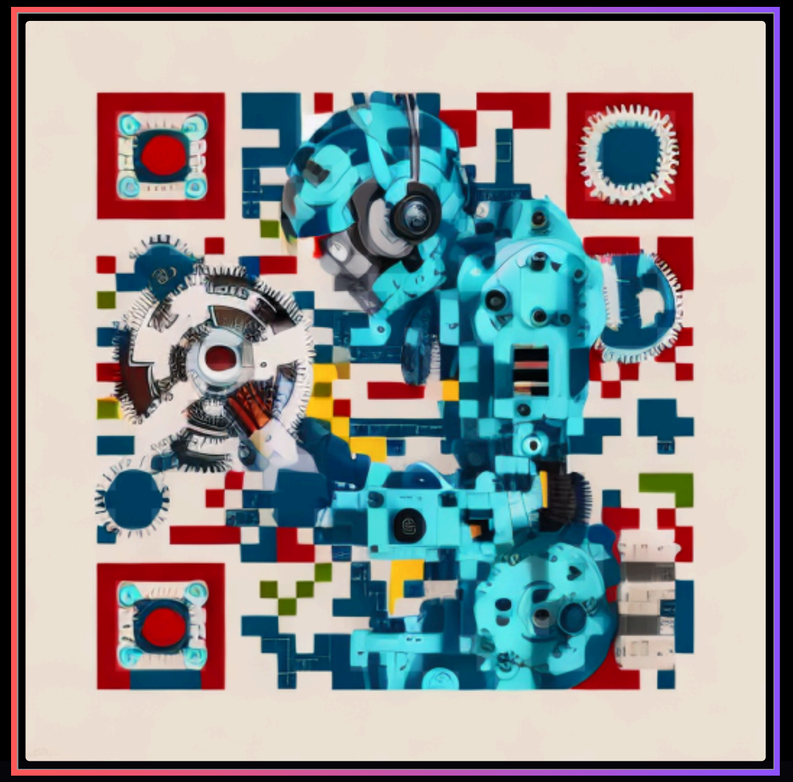
in/pathakpratik/

zpratikpathak

ARCHITECTURE · PERIODIC CONSOLIDATION

How dreaming works





Questions?

"The future of software engineering is not learning how to write better English prompts to an AI chatbot. It is writing better invariant test suites and context guardrails so the AI prompts itself until the software is provably correct."